Database Management Systems

Rapid-Revision One-Liners

IBPS SO (IT OFFICER)

COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY

Asabhya Maanav

2026 EDITION

VERSION 1.0

Table of Contents

Database Management Systems - One-Liners

Introduction and Basic Concepts

1. The **4** core operations of any DBMS are **CRUD**: **Create, Read, Update, Delete**.
2. The processing chain has **3** stages: **Data -> Information -> Knowledge**.
3. There are **2** key terms often confused: **data** (raw facts) and **information** (processed, meaningful data).
4. "Data about data" has **1** name - **metadata** - and it is stored in the **data dictionary**.
5. The **2** main rivals for storage are the **File Processing System** and the **DBMS**.
6. A file system suffers from **2** major flaws absent in a DBMS: **data redundancy** and **data inconsistency**.
7. The DBMS advantage memory hook "R-I-C-S" lists **4** wins: **Redundancy control, Integrity, Concurrency, Security**.
8. There are at least **5** disadvantages of a DBMS: **high cost, complexity, performance overhead, large size** and **single point of failure**.
9. A DBMS is **software** while a database is **data** - **2** distinct things students often merge.
10. Common DBMS application domains include **6+**: **banking, railways/airlines reservation, universities, e-commerce, telecom** and **HR/payroll**.
11. There are **2** broad benefits of constraints: **data integrity** and **consistency**.
12. The full system needing **both** parts is called a **database system** = **DBMS + the database**.

Database Architecture and Data Independence

1. The **ANSI/SPARC** architecture (year **1975**) defines **3** levels: **internal, conceptual** and **external**.
2. The **internal level** answers **1** question - **HOW** data is physically stored (files, indexes).
3. The **conceptual level** answers **1** question - **WHAT** data and relationships exist (community view).
4. The **external level** controls **1** thing - **WHO** sees which subset, implemented via **views**.
5. There is exactly **1** conceptual schema, **1** internal schema, but **many** external schemas per database.
6. There are **2** types of data independence: **physical** (easier) and **logical** (harder).
7. **Physical data independence** isolates **2** levels - changing **internal** without touching **conceptual**.
8. **Logical data independence** isolates **2** levels - changing **conceptual** without touching **external/apps**.
9. There are **2** mappings in the 3-schema model: **external/conceptual** and **conceptual/internal**.
10. There are **2** stable-vs-volatile concepts: **schema** (design, rarely changes) and **instance** (current data, changes often).
11. Data abstraction is provided through the same **3** levels: **physical, logical** and **view**.
12. The memory hook is "**P**hysical is **P**ossible (easy), **L**ogical is **L**aborious (hard)" - **2** independences ranked.

Database Users, DBA and Languages

1. There are **5** types of database users: **naive, application programmers, sophisticated, specialized** and the **DBA**.
2. The **DBA** has at least **4** duties: **schema definition, authorization (GRANT/REVOKE), backup/recovery** and **tuning**.
3. SQL has **5** sub-languages: **DDL, DML, DQL, DCL** and **TCL**.
4. **DDL** owns **5** commands: **CREATE, ALTER, DROP, TRUNCATE** and **RENAME**.
5. **DML** owns **3** commands: **INSERT, UPDATE** and **DELETE**.
6. **DCL** owns **2** commands: **GRANT** and **REVOKE**.
7. **TCL** owns **3** commands: **COMMIT, ROLLBACK** and **SAVEPOINT**.
8. **DQL** owns **1** command: **SELECT**.
9. DML comes in **2** flavours: **procedural** (relational algebra) and **non-procedural** (SQL, relational calculus).
10. The **data dictionary** stores **1** category of content - **metadata** - and is **self-describing**.
11. The database engine splits into **2** big parts: the **query processor** and the **storage manager**.
12. The **storage manager** contains **4** modules: **authorization/integrity, transaction, buffer** and **file** managers.
13. The **query processor** contains **3** parts: **DDL interpreter, DML compiler/optimizer** and **query evaluation engine**.

Data Models

1. There are **6+** classic data models: **hierarchical, network, relational, object-oriented, object-relational** and **ER**.
2. The **hierarchical model** uses **1** structure - a **tree** - and supports only **1:N** (single parent).
3. The **network model** uses **1** structure - a **graph** - and supports **M:N** via the **owner-member set**.
4. The **relational model** was given by **1** person - **E. F. Codd** - in the year **1970**.
5. The **ER model** was given by **1** person - **Peter Chen** - in the year **1976**.
6. The hierarchical model was first realized in **1** product - **IBM IMS** (year **1968**).
7. The network model was standardized by **1** body - **CODASYL DBTG**.
8. There are **3** data-model abstraction categories: **high-level (conceptual), representational** and **low-level (physical)**.
9. The **object-relational model** blends **2** worlds - **relational tables** plus **OO features** (e.g. PostgreSQL).
10. A **DBTG set** has exactly **1** owner record type and **1 or more** member record types.
11. To model **M:N** in a network model you need **2** 1:N sets joined by **1** dummy record.
12. The relational model rests on **2** mathematical pillars: **set theory** and **predicate logic**.

Entity-Relationship (ER) and EER Model

1. The ER model has **3** building blocks: **entities, attributes** and **relationships**.
2. There are **2** entity types: **strong** (own key, single rectangle) and **weak** (no key, double rectangle).
3. A weak entity is identified by **2** things: the **owner's primary key** plus its own **partial key**.
4. Attributes split by divisibility into **2**: **simple (atomic)** and **composite**.
5. Attributes split by count into **2**: **single-valued** and **multi-valued** (double oval).
6. Attributes split by storage into **2**: **stored** and **derived** (dashed oval).
7. The ERD shape code: **rectangle = entity, oval = attribute, diamond = relationship, underline = key** - **4** symbols.
8. "Double means many" maps **3** shapes: **double oval = multivalued, double rectangle = weak, double diamond = identifying**.
9. Relationship **degree** has **3** common values: **unary (1), binary (2)** and **ternary (3)**.
10. **Mapping cardinality** has **4** ratios: **1:1, 1:N, M:1** and **M:N**.
11. **Participation** has **2** types: **total** (double line, mandatory) and **partial** (single line, optional).
12. A relationship to the same entity set has **1** name - **recursive (unary)** - and uses **role names**.
13. EER adds **3** features: **specialization/generalization, aggregation** and **categorization (union)**.
14. **Specialization** is **1** direction - **top-down** (split); **generalization** is the opposite - **bottom-up** (combine).
15. Generalization hierarchies have **2** constraint pairs: **disjoint vs overlapping** and **total vs partial**.
16. **Aggregation** solves **1** ER limitation - representing a **relationship between an entity and another relationship**.
17. ER-to-table mapping uses **1** golden rule: **M:N becomes a new table**, **1:N just borrows a foreign key**.
18. A multivalued attribute always maps to **1** thing - a **separate table** (owner PK + the value).

Keys

1. There are **6+** key types: **super, candidate, primary, alternate, foreign** and **composite**.
2. The key hierarchy nests **3** levels: **primary (subset of) candidate (subset of) super key**.
3. A **candidate key** is **1** special super key - the **minimal** one.
4. A **primary key** enforces **2** rules: **NOT NULL** and **UNIQUE**.
5. **Alternate keys** are **all candidate keys except 1** (the chosen primary key).
6. A **foreign key** references **1** thing - the **primary key of another (or the same) relation**.
7. A **composite key** is made of **2 or more** attributes.
8. A **surrogate key** is **1** artificial, **system-generated** identifier (e.g. auto-increment id).
9. Only **1** primary key is allowed per relation, but a table may have **multiple** candidate keys.

Relational Model and Integrity Constraints

1. A relation's structure uses **4** terms: **relation (table), tuple (row), attribute (column)** and **domain (value set)**.

2. There are **2** size measures: **degree = number of columns** and **cardinality = number of rows**.
3. A relation has **2** parts: **schema** (structure) and **instance** (current tuples).
4. Relations obey **3** key properties: **no duplicate tuples, order immaterial** and **atomic cell values**.
5. There are **5+** integrity constraints: **domain, key, entity integrity, referential integrity** plus **NOT NULL/UNIQUE/CHECK/DEFAULT**.
6. **Entity integrity** enforces **1** rule: a **primary key cannot be NULL**.
7. **Referential integrity** enforces **1** rule: a **foreign key must match an existing PK or be NULL**.
8. Referential actions number **4**: **CASCADE, SET NULL, SET DEFAULT** and **RESTRICT/NO ACTION**.
9. A **NULL** means **1** of two things: **unknown** or **not applicable** - never zero or blank.
10. The comparison `NULL = NULL` yields **1** result - **UNKNOWN** - not TRUE.
11. A primary key can never be NULL, but a foreign key **can** be NULL - **2** opposite rules.

Relational Algebra (Procedural)

1. Relational algebra is **procedural** and has **6** fundamental operators: **select, project, union, set-difference, Cartesian product** and **rename**.
2. **Selection (sigma)** filters **1** dimension - **rows** (horizontal); **projection (pi)** filters **columns** (vertical).
3. **Projection** does **1** extra thing selection never does - **removes duplicate** tuples.
4. There are **3** unary operators: **selection (sigma), projection (pi)** and **rename (rho)**.
5. Set operations need **1** precondition - **union compatibility** (same arity + domains).
6. There are **4** binary set/product operators: **union, intersection, set-difference** and **Cartesian product**.
7. **Union and intersection** are **commutative**, but **set-difference** is **NOT** - a **2**-vs-**1** split.
8. For `R x S`, degree = **sum of degrees** and cardinality = **product of cardinalities**.
9. There are **5+** join types: **theta, equi, natural, outer** and **semi** join.
10. **Natural join** does **1** clean-up - it **removes the duplicate common column** after equi-join.
11. **Outer joins** come in **3** kinds: **left, right** and **full** - all padding unmatched rows with **NULL**.
12. **Division (R/S)** answers **1** type of query - **"for all / every"** questions.
13. The fundamental-operator mnemonic "**SPUDCR**" packs **6** ops: **Select, Project, Union, Difference, Cross, Rename**.
14. Intersection is **derived**, expressible as **1** formula: `R n S = R - (R - S)`.

Relational Calculus (Non-Procedural)

1. Relational calculus is **non-procedural** and comes in **2** forms: **Tuple (TRC)** and **Domain (DRC)**.
2. **TRC** variables range over **1** thing - **tuples** - written `{ t | P(t) }`.
3. **DRC** variables range over **1** thing - **domain values**, written `{ <x1,...,xn> | P(...) }`.
4. Calculus uses **2** quantifiers: **EXISTS (there-exists)** and **FOR ALL (universal)**.
5. A **safe expression** guarantees **1** property - a **finite** result from the **active domain**.
6. **Codd's theorem** states **2** languages are equivalent: **relational algebra** and **safe relational calculus**.
7. The big contrast: **algebra = HOW (procedural)** vs **calculus = WHAT (declarative)** - **2** philosophies.
8. SQL is closer to **1** of them - **relational calculus** (declarative) - though optimizers use **algebra**.

SQL Basics and Data Types

1. **SQL** was originally named **SEQUEL**, created by **IBM** for **System R** in the year **1974**.
2. The first ANSI SQL standard has **1** name - **SQL-86**; later milestones include **SQL-92, SQL:1999, SQL:2003**.
3. SQL uses **bag (multiset)** semantics by default, keeping **duplicates** unless **1** keyword - **DISTINCT** - is added.
4. SQL data types fall into **4** groups: **numeric, character, date/time** and **others (BLOB/BOOLEAN)**.
5. There are **2** confusable string types: **CHAR(n)** (fixed-length, padded) and **VARCHAR(n)** (variable-length).
6. Numeric types include **6+**: **INT, SMALLINT, BIGINT, DECIMAL, FLOAT** and **REAL**.
7. Date/time types include **4**: **DATE, TIME, TIMESTAMP** and **INTERVAL**.
8. SQL keywords are **case-insensitive**, but stored **data** is **case-sensitive** - **2** different behaviours.

SQL DDL (Data Definition Language)

1. DDL has **5** commands: **CREATE, ALTER, DROP, TRUNCATE** and **RENAME**.
2. There are **3** delete-style commands compared often: **DROP, TRUNCATE** and **DELETE**.
3. **TRUNCATE** is **DDL** with **1** key trait - it **cannot be rolled back** (auto-commit) and has no `WHERE`.
4. **DELETE** is **DML** with **2** abilities **TRUNCATE** lacks: a **WHERE clause** and **ROLLBACK**.
5. **DROP** removes **3** things at once: the **rows, structure** and **the table itself**.
6. `ALTER TABLE` can perform **3+** actions: **ADD, MODIFY** and **DROP COLUMN**.
7. Column constraints definable in DDL number **5+**: **PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK** and **DEFAULT**.
8. **TRUNCATE** also does **1** extra thing **DELETE** does not - it **resets the identity/auto-increment** counter.

SQL DML and DQL (Querying)

1. The SELECT clause **writing order** is **6** parts: **SELECT-FROM-WHERE-GROUP BY-HAVING-ORDER BY**.
2. The SELECT **execution order** differs, starting with **1** clause - **FROM** - then WHERE, GROUP BY, HAVING, SELECT, ORDER BY.
3. **WHERE** filters **1** thing (rows, pre-grouping); **HAVING** filters **another** (groups, post-aggregation).
4. There are **5** standard aggregate functions: **COUNT, SUM, AVG, MIN** and **MAX**.
5. Aggregates ignore **NULLs** with exactly **1** exception - **COUNT(*)** - which counts every row.
6. The `LIKE` operator uses **2** wildcards: **%** (any string) and `_` (single character).
7. Null testing needs **2** special operators: **IS NULL** and **IS NOT NULL** (never `= NULL`).
8. Range/membership filters number **3**: **BETWEEN, IN** and **LIKE**.
9. **DISTINCT** has **1** job - **removing duplicate** rows from the result.
10. `ORDER BY` defaults to **1** direction - **ascending (ASC)** - and uses **DESC** for descending.
11. Comparison operators number **6**: **=, <>(or !=), <, >, <=** and **>=**.
12. An aggregate can appear in **HAVING** but **never** in **WHERE** - **1** classic restriction.

Joins in SQL

1. SQL joins number **6** types: **INNER, LEFT, RIGHT, FULL OUTER, CROSS** and **SELF** join.
2. An **INNER JOIN** keeps **1** kind of row - only the **matching** rows from both tables.
3. A **LEFT OUTER JOIN** keeps **all left** rows plus matches, padding gaps with **NULL**.
4. A **FULL OUTER JOIN** behaves like a row-set **union** of LEFT and RIGHT joins.
5. A **CROSS JOIN** of **m** and **n** rows yields exactly **m*n** rows (Cartesian product).
6. A **SELF JOIN** uses **1** table joined to **itself** via **table aliases**.
7. The memory hook maps **2** joins: **INNER = intersection** and **FULL OUTER = union**.
8. A join condition uses **1** keyword - **ON** - while a natural join needs **no** condition.

Subqueries, Views and Set Operators

1. Subqueries come in **3** types: **single-row, multi-row** and **correlated**.
2. A **multi-row** subquery needs **3** operators: **IN, ANY** and **ALL**.
3. A **correlated** subquery runs **1** way - **once per outer row** (referencing the outer query).
4. **EXISTS** returns **TRUE** when the subquery returns **at least 1** row; **NOT EXISTS** is the inverse.
5. Set operators number **4**: **UNION, UNION ALL, INTERSECT** and **MINUS/EXCEPT**.
6. **UNION** removes duplicates while **UNION ALL** keeps them - **2** behaviours, the latter **faster**.
7. Set operators require **1** precondition - the SELECTs must be **union-compatible**.
8. A **view** is **1** thing - a **virtual table** storing **no data** (unless **materialized**).
9. A view is **updatable** only if it avoids **4+** features: **aggregates, DISTINCT, GROUP BY** and **joins**.
10. Views give **3** benefits: **security, simplicity** and **logical data independence**.

Indexes, Sequences, DCL and TCL

1. Indexes split into **2** types: **clustered** (physical order, **1 per table**) and **non-clustered** (**many per table**).
2. A **clustered index** is limited to **1** per table because rows can be physically sorted only **one** way.
3. A `PRIMARY KEY` by default creates **1** index type - a **clustered** index.
4. A **sequence** generates **1** thing - **unique numeric values** - via **NEXTVAL** and **CURRVAL**.
5. DCL has **2** commands: **GRANT** (give) and **REVOKE** (remove) privileges.
6. `WITH GRANT OPTION` adds **1** power - letting the grantee **re-grant** the privilege.
7. TCL has **3** commands: **COMMIT** (permanent), **ROLLBACK** (undo) and **SAVEPOINT** (partial marker).
8. `ROLLBACK TO savepoint` undoes work back to **1** marker without discarding the whole transaction.

PL/SQL, Triggers and Cursors

1. **PL/SQL** is **1** thing - Oracle's **procedural extension** to SQL (loops, IF, exceptions).
2. There are **3** named program units compared: **procedure, function** and **trigger**.
3. A **function** returns exactly **1** value and can be used inside a **SELECT**.
4. A **procedure** returns **0..many** values via **OUT** parameters and is run with **CALL/EXEC**.
5. A **trigger** fires **automatically** on **3** events: **INSERT, UPDATE** and **DELETE**.
6. Triggers have **2** timings (**BEFORE/AFTER**) and **2** levels (**ROW/STATEMENT**).
7. A **cursor** processes results **1 row at a time** and comes in **2** kinds: **implicit** and **explicit**.
8. An explicit cursor uses **3** operations: **OPEN, FETCH** and **CLOSE**.
9. An **assertion** is **1** thing - a **constraint over the whole database** that must always hold.

Functional Dependencies

1. Normalization removes **3** anomalies: **insertion, deletion** and **update** anomalies.
2. An FD `X -> Y` has **2** sides: **X** (determinant) and **Y** (dependent).
3. FDs split into **2** types: **trivial** (Y is a subset of X) and **non-trivial**.
4. **Armstrong's axioms** number **3** primary rules: **reflexivity, augmentation** and **transitivity** (mnemonic "RAT").
5. Armstrong's axioms have **2** quality guarantees: they are **sound** and **complete**.
6. Derived FD rules number **4**: **union, decomposition, pseudo-transitivity** and **composition**.
7. **Augmentation** states **1** thing: if `X -> Y` then `XZ -> YZ`.
8. **Transitivity** states **1** thing: if `X -> Y` and `Y -> Z` then `X -> Z`.
9. **Attribute closure (X+)** has **3** uses: test a **super key**, test an **FD**, and find **candidate keys**.
10. A **minimal cover** satisfies **3** conditions: **single-attribute RHS, no redundant FD** and **no extraneous LHS attribute**.
11. Two FD sets are **equivalent** when **1** condition holds - their **closures are equal (F+ = G+)**.
12. The **union rule** states: if `X -> Y` and `X -> Z` then **X -> YZ** (combining **2** FDs into **1**).

Candidate Keys and Decomposition

1. Attributes appearing **only on the LHS** must be in **every** candidate key.
2. Attributes appearing **only on the RHS** are in **no** candidate key.
3. Attributes in **no FD at all** must be in **every** candidate key.
4. Attributes split into **2** classes: **prime** (in some candidate key) and **non-prime** (in none).
5. There are **2** desirable decomposition properties: **lossless join** and **dependency preservation**.
6. A binary decomposition is **lossless** if **1** condition holds: the **common attributes form a key of at least one part**.
7. **Lossless join** must always hold; **dependency preservation** is desirable but **not always achievable** - a **2**-tier priority.
8. A decomposition test for losslessness checks **1** thing: `(R1 INTERSECT R2)` determines **R1 or R2**.

Normal Forms

1. The normal-form ladder has **6** rungs: **1NF, 2NF, 3NF, BCNF, 4NF** and **5NF**.
2. **1NF** requires **1** thing - all attribute values are **atomic** (no repeating groups).

3. **2NF** removes **1** problem - **partial dependency** of non-prime attributes on part of a key.
4. **3NF** removes **1** problem - **transitive dependency** of a non-prime attribute on a key.
5. **3NF** is satisfied if for every FD `X -> A`, **X is a super key OR A is a prime attribute** - **2** escapes.
6. **BCNF** allows only **1** escape - for every non-trivial FD `X -> A`, **X must be a super key**.
7. **4NF** removes **1** dependency type - the **multivalued dependency (MVD)**.
8. **5NF (PJNF)** removes **1** dependency type - the **join dependency (JD)**.
9. The FD-based normal forms number **3**: **2NF, 3NF** and **BCNF**.
10. The most-tested fact: **3NF guarantees both lossless join AND dependency preservation**, but **BCNF may sacrifice dependency preservation**.
11. A relation with a **single-attribute candidate key** and all attributes prime is automatically in **3NF/BCNF**.
12. Strictness order ranks **6** forms: **1NF < 2NF < 3NF < BCNF < 4NF < 5NF**.
13. Every **FD** is also an **MVD**, but not every MVD is an FD - a **1**-way implication.
14. **2NF** is irrelevant when every candidate key is a **single attribute** (no partial dependency possible).

Transactions and ACID

1. A transaction is **1** logical unit of work with **2** primitive operations: **read(X)** and **write(X)**.
2. A transaction passes through **5** states: **active, partially committed, committed, failed** and **aborted**.
3. The **ACID** properties number **4**: **Atomicity, Consistency, Isolation** and **Durability**.
4. **Atomicity** guarantees **1** rule - **all-or-nothing** (no partial transaction).
5. **Durability** guarantees **1** rule - committed changes **survive crashes**.
6. **Isolation** guarantees **1** illusion - concurrent transactions appear to run **serially**.
7. ACID is split across **2** managers: **recovery** (Atomicity + Durability) and **concurrency control** (Isolation).
8. A transaction enters the **failed** state from **1** prior state - **active** - after an error.
9. Once a transaction is **committed**, its effect cannot be undone - **1** irreversible state.
10. Transaction control uses **2** terminal commands: **COMMIT** and **ROLLBACK**.

Schedules and Serializability

1. Schedules split into **2** types: **serial** (no interleaving) and **non-serial** (interleaved).
2. Serializability has **2** notions: **conflict serializability** and **view serializability**.
3. Two operations conflict if **3** conditions hold: **different transactions, same data item** and **at least one write**.
4. Conflicting operation pairs number **3**: **read-write, write-read** and **write-write** (R-R never conflicts).
5. A schedule is **conflict serializable** iff **1** condition - its **precedence graph is acyclic**.
6. The relationship is **1**-way: **conflict-serializable is a subset of view-serializable**.
7. The extra view-serializable schedules contain **1** feature - a **blind write** (write without prior read).
8. Recoverability has **3** levels: **recoverable, cascadeless** and **strict**.
9. The strictness order ranks **3**: **strict (subset of) cascadeless (subset of) recoverable**.
10. A **cascadeless** schedule prevents **1** problem - **cascading rollback** (no dirty reads).
11. A **strict** schedule forbids **2** actions on uncommitted data: **reading AND writing** it.
12. A precedence graph has nodes = **transactions** and an edge for each conflicting pair - **2** components.

Concurrency Control Protocols

1. Concurrency problems number **4**: **lost update, dirty read, unrepeatable read** and **phantom read**.
2. A **dirty read** is reading **1** kind of value - an **uncommitted** one (W-R conflict).
3. A **lost update** arises from **2** writes on the same item, one overwriting the other.
4. There are **2** lock modes: **shared (S)** for reading and **exclusive (X)** for writing.
5. Lock compatibility allows only **1** combination - **S with S**; **X** is incompatible with everything.
6. **Two-Phase Locking (2PL)** has **2** phases: **growing** (acquire only) and **shrinking** (release only).
7. **2PL** guarantees **1** property - **conflict serializability** - but **not** deadlock-freedom.

8. 2PL variants number **4**: **basic, strict, rigorous** and **conservative (static)**.
9. **Strict 2PL** holds **1** lock type until commit - all **exclusive (X)** locks.
10. **Rigorous 2PL** holds **all** locks (both **S and X**) until commit.
11. **Conservative 2PL** is the only variant that is **deadlock-free** (acquires all locks upfront).
12. The **lock point** is **1** moment - when the **last lock is acquired** (end of growing phase).
13. Deadlock handling has **3** strategies: **prevention, detection** and **timeout**.
14. Timestamp-based deadlock prevention uses **2** schemes: **Wait-Die** and **Wound-Wait**.
15. In **Wait-Die**, the **older** transaction **waits** and the younger **dies (restarts)**.
16. In **Wound-Wait**, the **older** transaction **wounds (aborts)** the younger, which then **waits**.
17. **Timestamp ordering** assigns each item **2** stamps: **read-TS** and **write-TS**.
18. **Thomas' Write Rule** does **1** optimization - it **ignores obsolete (outdated) writes**.
19. **Optimistic (validation) concurrency** runs in **3** phases: **read, validation** and **write**.
20. **Multiple granularity** locking adds **3** intention locks: **IS, IX** and **SIX**.
21. Protocols that **cannot** deadlock number **3**: **conservative 2PL, timestamp ordering** and **optimistic**.
22. **Starvation** is solved by **1** technique - **aging** (or FIFO lock granting).

Database Recovery

1. Failures classify into **3** types: **transaction failure, system crash** and **disk/media failure**.
2. A typical log record holds **4** fields: **transaction-id, data-item, old value** and **new value**.
3. Recovery techniques number **2**: **deferred update** and **immediate update**.
4. **Deferred update** needs only **1** action on recovery - **REDO** (no UNDO).
5. **Immediate update** needs **2** actions on recovery - **UNDO** and **REDO**.
6. **Write-Ahead Logging (WAL)** enforces **1** rule - the **log record reaches disk before the data page**.
7. A **checkpoint** shortens recovery by limiting the scan to **1** point - the **last checkpoint**.
8. **Shadow paging** keeps **2** page tables: **current** and **shadow (old)** - needing **no log**.
9. **ARIES** runs in **3** passes: **Analysis, Redo** and **Undo**.
10. ARIES relies on **3** concepts: **WAL, LSNs** and **fuzzy checkpoints**.
11. The log is stored on **1** kind of medium - **stable (non-volatile) storage**.
12. **Deferred update** is also called **NO-UNDO/REDO**, while immediate is **UNDO/REDO** - **2** names.
13. A system crash loses **1** thing - **main memory** - while disk failure loses **non-volatile storage**.
14. Recovery and concurrency together protect **2** ACID properties each: **A+D** by recovery, **I** by concurrency.

File Organization and Indexing

1. The storage hierarchy spans **6** levels: **registers, cache, main memory, SSD, magnetic disk** and **tape**.
2. Storage divides into **3** tiers: **primary (volatile), secondary** and **tertiary (non-volatile)**.
3. File organizations number **3**: **heap (unordered), sequential (ordered)** and **hash**.
4. A **heap file** gives **1** complexity for search - **O(n)** - but fast inserts.
5. A **hash file** gives **1** complexity for equality search - **O(1)** - but is poor for range queries.
6. Index types by ordering number **2**: **primary/clustering** (on ordering key) and **secondary** (non-ordering).
7. Index density has **2** types: **dense** (entry per record) and **sparse** (entry per block).
8. A key rule: a **primary index can be sparse**, but a **secondary index must be dense**.
9. There can be **at most 1** primary/clustering index but **many** secondary indexes per file.
10. A **multilevel index** is **1** idea - an **index built on top of another index** - leading to B+ trees.
11. Indexing trades **2** things: faster **reads** for slower **writes** (INSERT/UPDATE/DELETE) plus extra storage.
12. An index entry has **2** parts: a **search-key value** and a **pointer** to the record/block.

B-Tree, B+ Tree and Hashing

1. A **B-tree of order m** has at most **m** children and at most **m-1** keys per node.

2. A non-root internal B-tree node has at least **ceil(m/2)** children - **1** minimum-occupancy rule.
3. A B-tree keeps **all leaves at the same level** - **1** balance guarantee.
4. B-tree search/insert/delete all cost **O(log n)** - **1** logarithmic complexity.
5. The big B-tree vs B+ tree difference: **B+ tree stores data only in leaves**, B-tree stores data in **all** nodes.
6. A **B+ tree** links its **leaf nodes** as **1** structure - a **linked list** for range scans.
7. Range queries are faster in **1** of them - the **B+ tree** (sequential linked leaves).
8. A B+ tree has **lower height / higher fan-out** than a B-tree of the same order - **1** efficiency edge.
9. The memory hook is "**B+ = data at the Bottom (leaves)**" - **1** mnemonic.
10. **Hashing** maps a key via **1** function `h(key)` to a **bucket**, ideally **O(1)**.
11. A **collision** is **1** event - **2 keys mapping to the same bucket**.
12. Collision resolution methods number **3**: **chaining, open addressing** and **overflow buckets**.
13. Hashing has **2** families: **static** (fixed buckets) and **dynamic** (grows/shrinks).
14. **Static hashing** suffers from **1** problem as data grows - **overflow chains**.
15. **Extendible hashing** uses **3** concepts: a **directory, global depth** and **local depth**.
16. **Extendible hashing** doubles **1** structure on overflow - the **directory** - splitting one bucket.
17. **Linear hashing** needs **no directory** and splits buckets in **1** order - **round-robin**.
18. Hashing wins for **equality (O(1))** but loses for **range queries** - use **B+ trees** for ranges - **2** opposite use-cases.

Distributed Databases

1. A distributed database appears as **1** logical database spread across **many** physical sites.
2. Distributed transparencies number **3**: **location, fragmentation** and **replication** transparency.
3. Fragmentation has **3** types: **horizontal (rows), vertical (columns)** and **hybrid/mixed**.
4. The **CAP theorem** allows only **2 of 3** guarantees: **Consistency, Availability** and **Partition-tolerance**.
5. The CAP theorem was proposed by **1** person - **Eric Brewer** (year **2000**).
6. Correct fragmentation must satisfy **3** rules: **completeness, reconstruction** and **disjointness**.
7. **Two-Phase Commit (2PC)** ensures atomic distributed commit in **2** phases: **prepare/voting** and **commit/decision**.
8. **2PC** uses **2** roles: **1 coordinator** and **multiple participants**.
9. Distributed DBs offer **3** advantages: **reliability, availability** and **local autonomy**.
10. **Replication** improves **1** thing (availability) but raises **1** cost (update synchronization) - a **2**-sided trade-off.
11. **Horizontal fragmentation** uses **1** operation - **selection** (splitting rows by predicate).
12. **Vertical fragmentation** uses **1** operation - **projection** (splitting columns).

Data Warehousing and OLAP

1. A data warehouse has **4** traits (Inmon): **subject-oriented, integrated, time-variant** and **non-volatile**.
2. The **2** opposed systems are **OLTP** (transactional) and **OLAP** (analytical).
3. Data-warehouse schemas number **3**: **star, snowflake** and **fact-constellation (galaxy)**.
4. A **star schema** has **1** central **fact table** surrounded by **multiple denormalized dimension** tables.
5. A **snowflake schema** differs by **1** trait - its **dimension tables are normalized**.
6. OLAP operations number **5**: **roll-up, drill-down, slice, dice** and **pivot (rotate)**.
7. **Roll-up** aggregates up a hierarchy; **drill-down** is the reverse - **2** opposite operations.
8. **Slice** fixes **1** dimension while **dice** fixes **multiple** dimensions (a sub-cube).
9. **ETL** has **3** stages: **Extract, Transform** and **Load**.
10. A **fact table** stores **2** things: **numeric measures** and **foreign keys** to dimensions.
11. A **data mart** is **1** thing - a **subset of a warehouse** for a single department.
12. OLAP server types number **3**: **ROLAP, MOLAP** and **HOLAP**.
13. **OLTP** is **normalized (3NF)** while **OLAP** is **denormalized (star)** - a **2**-design contrast.
14. A **data cube** generalizes a table to **n** dimensions for multidimensional analysis.

NoSQL and Query Optimization

1. **NoSQL** ("Not Only SQL") has **4** types: **key-value, document, column-family** and **graph**.
2. NoSQL examples map **4** stores: **Redis (KV), MongoDB (document), Cassandra (column)** and **Neo4j (graph)**.
3. **BASE** has **3** parts: **Basically Available, Soft state** and **Eventual consistency**.
4. NoSQL favours **1** model (BASE) over the relational **1** model (ACID) - a **2**-way contrast.
5. Query processing has **3** steps: **parsing/translation, optimization** and **evaluation**.
6. Query optimization has **2** approaches: **cost-based** and **heuristic (rule-based)**.
7. The main heuristic does **1** thing - **push selections and projections down** the query tree.
8. The optimizer picks **1** thing - the lowest-cost **query execution plan** (I/O + CPU).
9. A **materialized view** differs from a normal view by **1** trait - it **physically stores** data.
10. Cost estimation reads statistics from **1** source - the **data dictionary/catalog**.
11. A **graph database** stores **2** elements: **nodes** and **edges (relationships)**.
12. MongoDB stores documents in **1** format - a **JSON/BSON-like** structure.

Pioneers, Years and Landmark Numbers

1. The **relational model** was introduced by **E. F. Codd** in the year **1970**.
2. The **ER model** was introduced by **Peter Chen** in the year **1976**.
3. The **ANSI/SPARC 3-schema** architecture dates to the year **1975**.
4. **SQL (as SEQUEL)** was built at **IBM** for **System R** in the year **1974**.
5. The **B-tree** was invented by **Bayer and McCreight** in the year **1972**.
6. The term **ACID** was coined by **Harder and Reuter** in the year **1983**.
7. The **hierarchical IBM IMS** system dates to the year **1968**.
8. **BCNF** was given by **Boyce and Codd** in the year **1974**.
9. **4NF** (multivalued dependency) was given by **Ronald Fagin** in the year **1977**.
10. **5NF** (join dependency) was also given by **Ronald Fagin** in the year **1979**.
11. **Codd's rules** number **13**, conveniently numbered **Rule 0 to Rule 12** (year **1985**).
12. **Two-Phase Locking** was formalized by **Eswaran et al.** in the year **1976**.
13. The first SQL standard has **1** name - **SQL-86** (ANSI, ratified ISO **1987**).
14. Major SQL standards number **5+**: **SQL-86, SQL-92, SQL:1999, SQL:2003** and **SQL:2008**.
15. The first commercial SQL RDBMS was **Oracle**, released in the year **1979**.
16. Two pioneering relational prototypes were **System R (IBM)** and **Ingres (Berkeley)**.
17. The **CAP theorem** was conjectured by **Eric Brewer** in the year **2000** and proven in **2002**.
18. The **network model** standard came from **1** body - **CODASYL DBTG**.

High-Yield Comparison Traps

1. There are **2** size terms swapped by examiners: **degree (columns)** and **cardinality (rows)**.
2. There are **2** algebra operators swapped: **selection (sigma, rows)** and **projection (pi, columns)**.
3. There are **2** delete commands contrasted: **DELETE (DML, rollback)** and **TRUNCATE (DDL, no rollback)**.
4. There are **2** keys contrasted: **primary key (never NULL)** and **foreign key (may be NULL)**.
5. There are **2** integrities contrasted: **entity (PK not NULL)** and **referential (FK matches PK)**.
6. There are **2** normal forms contrasted: **2NF (removes partial)** and **3NF (removes transitive)**.
7. There are **2** strict forms contrasted: **3NF (prime exception allowed)** and **BCNF (no exception)**.
8. There are **2** string types contrasted: **CHAR (fixed)** and **VARCHAR (variable)**.
9. There are **2** filters contrasted: **WHERE (rows, pre-group)** and **HAVING (groups, post-aggregate)**.
10. There are **2** index orders contrasted: **clustered (1 per table)** and **non-clustered (many)**.
11. There are **2** trees contrasted: **B-tree (data in all nodes)** and **B+ tree (data in leaves, linked)**.
12. There are **2** hashings contrasted: **static (fixed buckets)** and **dynamic/extendible (adaptive)**.
13. There are **2** serializabilities contrasted: **conflict (subset of)** and **view** serializability.

14. There are **2** query languages contrasted: **algebra (procedural)** and **calculus (declarative)**.
15. There are **2** calculi contrasted: **TRC (tuple variables)** and **DRC (domain variables)**.
16. There are **2** old models contrasted: **hierarchical (tree, 1:N)** and **network (graph, M:N)**.
17. There are **2** EER directions contrasted: **specialization (top-down)** and **generalization (bottom-up)**.
18. There are **2** workloads contrasted: **OLTP (normalized, write-heavy)** and **OLAP (denormalized, read-heavy)**.
19. There are **2** consistency models contrasted: **ACID (relational)** and **BASE (NoSQL)**.
20. There are **2** design terms contrasted: **schema (stable design)** and **instance (current data)**.
21. There are **2** set operators contrasted: **UNION (removes duplicates)** and **UNION ALL (keeps them)**.
22. There are **2** program units contrasted: **function (returns exactly 1 value)** and **procedure (0..many)**.
23. There are **2** index densities contrasted: **dense (per record)** and **sparse (per block)**.
24. There are **2** independences contrasted: **physical (easy)** and **logical (hard)**.

Rapid-Fire Mixed Revision

1. **Codd's rules** total **13**, numbered **Rule 0 through Rule 12**.
2. The relational model rests on **2** foundations: **set theory** and **predicate logic**.
3. A **candidate key** is **1** minimal super key (no removable attribute).
4. Relational algebra has **6** fundamental operators (mnemonic **SPUDCR**).
5. A natural join with **0** common attributes becomes **1** thing - a **Cartesian product**.
6. A relation forbids **1** thing because it is a set - **duplicate tuples**.
7. **SQL DISTINCT** mimics **1** algebra operator - **projection** (duplicate removal).
8. Normalization removes **3** anomalies progressively: **insert, delete** and **update**.
9. A **weak entity** has **1** participation type in its identifying relationship - **total**.
10. An **M:N** relationship needs **1** extra table with a **composite key** of both PKs.
11. **COMMIT** makes changes permanent; **ROLLBACK** undoes to the last commit/savepoint - **2** opposite TCL actions.
12. A **trigger** differs from a procedure by **1** trait - it **fires automatically**.
13. The **buffer manager** controls **1** thing - which data stays in **main memory**.
14. Foreign-key delete actions number **4**: **CASCADE, SET NULL, SET DEFAULT** and **RESTRICT**.
15. A schedule with a **precedence-graph cycle** fails **1** test - **conflict serializability**.
16. A **serial** schedule is always **serializable** - **1** guaranteed property.
17. **Thomas' Write Rule** belongs to **1** protocol family - **timestamp ordering**.
18. **Lost update** and **dirty read** are both **write-related** concurrency problems.
19. **Hashing** best serves **equality** search; **B+ trees** best serve **range** search - **2** use-cases.
20. **Lossless join** is mandatory while **dependency preservation** is optional - a **2**-tier rule.
21. **Armstrong's axioms** are both **sound** and **complete** for FD inference.
22. The **data dictionary** stores **1** kind of content - **self-describing metadata**.
23. **1NF** forbids **2** attribute kinds in a cell: **multivalued** and **composite**.
24. In $R \times S$, **degrees add** and **cardinalities multiply** - **2** arithmetic rules.
25. **EXISTS** short-circuits at the **first** matching row, often beating **IN**.
26. **SQL** is closer to **1** formalism - **relational calculus** (declarative).
27. A **primary index** orders the data file; a **secondary index** does not - **2** behaviours.
28. **Two-Phase Commit (2PC)** guarantees **1** property - **atomic distributed commit**.
29. The **transaction** has **5** states and obeys **4** ACID properties - **2** key counts.
30. A **B+ tree** keeps all leaves at the **same level** and links them for **range** scans.